

Branch & Bound

Algoritmi di ottimizzazione [266SM]

Lorenzo Castelli

10 Aprile 2026



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**

Branch and bound uses a “divide and conquer” approach to explore the set of feasible integer solutions. However, instead of exploring the entire feasible set, it uses bounds on the optimal value to avoid certain parts of the set of feasible integer solutions.

Consider the problem

$$z = \max\{cx : x \in S\}$$

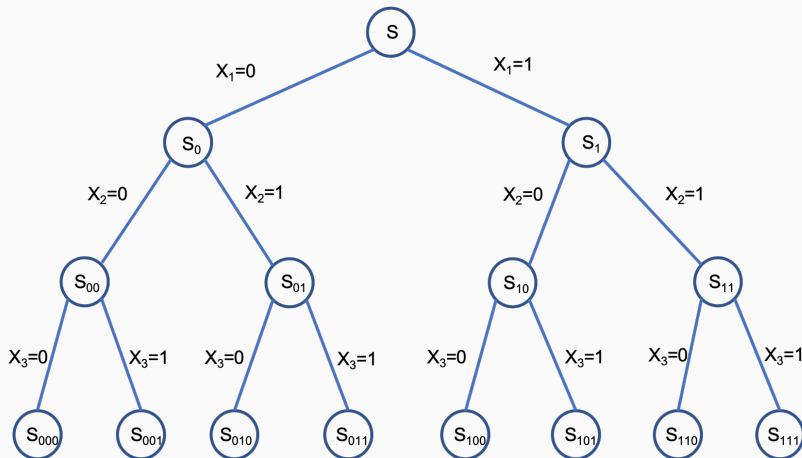
How can we break the problem into a series of subproblems that are easier, solve the smaller problems, and then put the information together again to solve the original problem?

Proposition 1

Let $S = S_1 \cup \dots \cup S_K$ be a decomposition of S into smaller sets, and let $z^k = \max\{cx : x \in S_k\}$ for $k = 1, \dots, K$. Then $z = \max_k z^k$

A typical way to represent such a divide and conquer approach is via an enumeration tree. For instance if $S \subseteq \{0, 1\}^3$, we might construct the enumeration tree shown in slide (3).

Enumeration tree



We first divide S into $S_0 = \{x \in S : x_1 = 0\}$ and $S_1 = \{x \in S : x_1 = 1\}$, then
 $S_{00} = \{x \in S_0 : x_2 = 0\} = \{x \in S : x_1 = x_2 = 0\}$,
 $S_{01} = \{x \in S_0 : x_2 = 1\} = \{x \in S : x_1 = 0, x_2 = 1\}$, and so on. Note that a leaf of the tree $S_{i_1 i_2 i_3}$ is nonempty if and only if $x = (i_1, i_2, i_3)$ is in S . Thus the leaves of tree correspond precisely to the point of $\{0, 1\}^3$ that one would examine if one carried out complete enumeration. By convention the tree is drawn upside down with the root at the top.

- Complete enumeration is totally impossible for most problems as soon as the number of variables in an integer program, or nodes in a graph exceeds 20 or 30. So we need to do more than just divide indefinitely.
- How can we use some bounds on the values of $\{z^k\}$ intelligently?
- First, how can we put together bound information?

Proposition

Let $S = S_1 \cup \dots \cup S_K$ be a decomposition of S into smaller sets, and let $z^k = \max\{cx : x \in S_k\}$ for $k = 1, \dots, K$, $\bar{z}^k$ be an upper bound on z^k and $\underline{z}^k$ be a lower bound on z^k . Then

$\bar{z} = \max_k \bar{z}^k$ is an upper bound on z

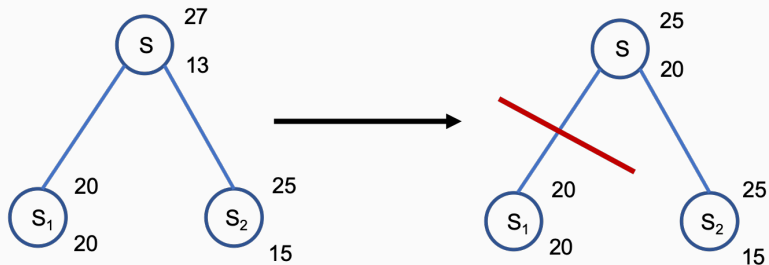
$\underline{z} = \max_k \underline{z}^k$ is a lower bound on z

What can be deduced about lower and upper bounds on the optimal value z and which sets need further examination in order to find an optimal value?

- In slide (8) we show a decomposition of S into two sets S_1 and S_2 as well as upper and lower bounds on the corresponding problems.
- We note that $\bar{z} = \max_k \bar{z}^k = \max\{20, 25\} = 25$.
- We note that $\underline{z} = \max_k \underline{z}^k = \max\{20, 15\} = 20$.
- We observe that as lower and upper bound on z^1 are equal, $z^1 = 20$, and there is no further reason to examine the set S_1 .

The branch S_1 of the enumeration tree can be pruned **by optimality**.

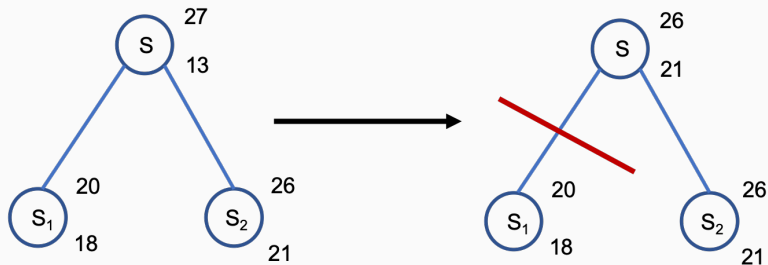
MAX



- In slide (10) we show a decomposition of S into two sets S_1 and S_2 as well as upper and lower bounds on the corresponding problems.
- We note that $\bar{z} = \max_k \bar{z}^k = \max\{20, 26\} = 26$.
- We note that $\underline{z} = \max_k \underline{z}^k = \max\{18, 21\} = 21$.
- We observe that as the optimal value has value at least 21, and the upper bound $\bar{z}^1 = 20$, no optimal solution can lie in the set S_1 .

The branch S_1 of the enumeration tree can be pruned **by bound**.

MAX



- In slide (12) we show a decomposition of S into two sets S_1 and S_2 as well as upper and lower bounds on the corresponding problems.
- We note that $\bar{z} = \max_k \bar{z}^k = \max\{24, 37\} = 37$.
- We note that $\underline{z} = \max_k \underline{z}^k = \max\{13, -\} = 13$.
- Here no other conclusions can be drawn and we need to explore both sets S_1 and S_2 further.

No pruning possible



MAX



How the bounds are to be obtained?

- The primal (lower) bounds are provided by feasible solutions.
- The dual (upper) bounds are provided by relaxation or duality.

Building an implicit enumeration algorithm based on the above ideas is now in principle a fairly straightforward task. There are, however, many questions that must be addressed before such an algorithm is well-defined.

- What relaxation or dual problem should be used to provide an upper bound?
- How should one choose between a fairly weak bound that can be calculated very rapidly and a stronger bound whose calculation takes a considerable time?
- How should a feasible region be separated into smaller regions
 $S = S_1 \cup \dots \cup S_K$?
- Should one separate into two or more parts?
- Should one use a fixed a priori rule for dividing up the set, or should the division evolve as a function of the bounds and solutions obtained en route?
- In what order should the subproblems be examined?
- Typically there is a list of active problems that have not yet been pruned. Should the next one be chosen on a basis of last-in first-out, of best/largest upper bound first, or of some totally different criterion?

Let's consider the following linear integer program

$$z = \max 4x_1 - x_2$$

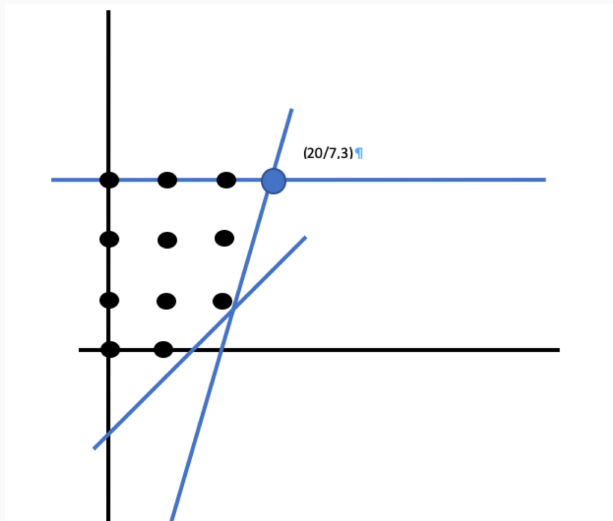
$$7x_1 - 2x_2 \leq 14$$

$$x_2 \leq 3$$

$$2x_1 - 2x_2 \leq 3$$

$$x \in \mathbb{Z}_+^2$$

- To obtain a first upper bound, we solve the linear programming relaxation in which the integrality constraints are dropped.
- We obtain an upper bound $\bar{z} = 59/7 = 8.43$ and a non-integral solution $(20/7, 3)$.
- Is there any straightforward way to find a feasible solution? In this case it is $(0, 0)$ and $\underline{z} = 0$.
- By convention, if no feasible solution is yet available, we take as lower bound $\underline{z} = -\infty$



Now because $\underline{z} < \bar{z}$, we need to divide or branch.

- How should we split up the feasible region?

One simple idea is to choose an integer variable which is nonzero and fractional in the linear programming solution, and split the problem into two about this fractional value. If $x_j = \bar{x}_j \notin \mathbb{Z}^1$, one can take:

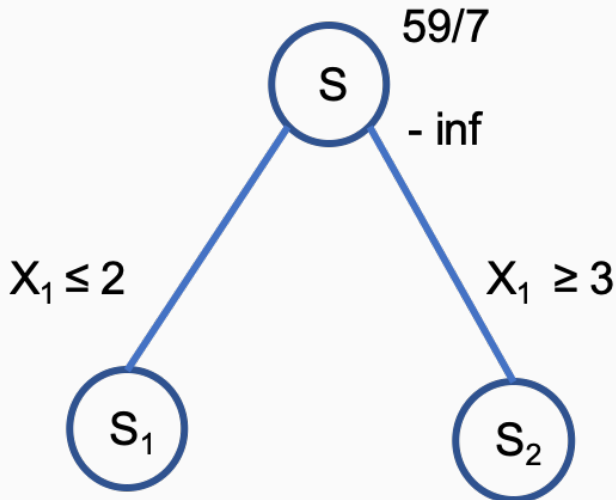
$$S_1 = S \cap \{x : x_j \leq \lfloor \bar{x}_j \rfloor\}$$

$$S_2 = S \cap \{x : x_j \geq \lceil \bar{x}_j \rceil\}$$

It is clear that $S = S_1 \cup S_2$ and $S_1 \cap S_2 = \emptyset$.

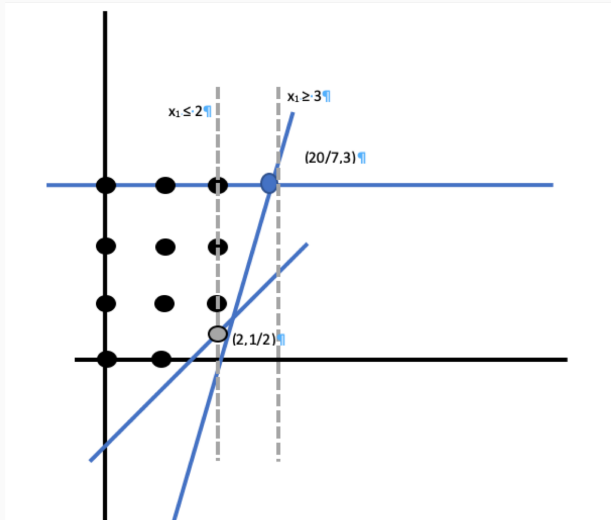
Another reason for this choice is that the solution $\bar{x}$ of $LP(S)$ is not feasible in either $LP(S_1)$ or $LP(S_2)$. This implies that (if there are not multiple LP solutions) $\max\{\bar{z}^1, \bar{z}^2\} < \bar{z}$, so the upper bound will strictly decrease.

Following this idea, as $\bar{x}_1 = 20/7 \notin \mathbb{Z}^1$, we take $S_1 = S \cap \{x : x_1 \leq 2\}$ and $S_2 = S \cap \{x : x_1 \geq 3\}$. The subproblems (nodes) that must still be examined are called **active**.





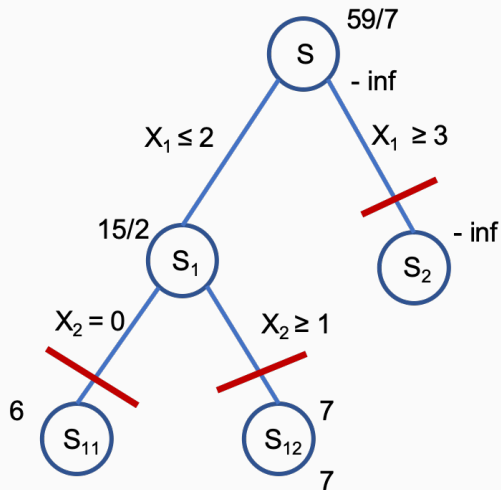
- Choosing a Node. The list of active problems (nodes) to be examined now contains S_1, S_2 . We arbitrarily choose S_1 .
- Reoptimizing. We solve the linear relaxation of S_1 . The results is $\bar{z}^1 = 15/2 = 7.5$, and $(\bar{x}_1^1, \bar{x}_2^1) = (2, 1/2)$.
- Branching. S_1 cannot be pruned, so using the same branching rule as before, we create two new nodes $S_{11} = S_1 \cap \{x : x_2 \leq 0\}$ and $S_{12} = S_1 \cap \{x : x_2 \geq 1\}$ and add them to the node list.



- Choosing a Node. The list of active problems (nodes) to be examined now contains S_2, S_{11}, S_{12} . We arbitrarily choose S_2 , and remove it by the node list.
- Reoptimizing. We solve the linear relaxation of S_2 . We see that the linear program is infeasible. Hence node S_2 is **pruned by infeasibility**.

- Choosing a Node. The node list now contains S_{11}, S_{12} . We arbitrarily choose S_{12} and remove it from the list.
- Reoptimizing. $S_{12} = S \cap \{x : x_1 \leq 2, x_2 \geq 1\}$. The resulting linear program has optimal solution $\bar{x}^{12} = (2, 1)$ and $\bar{z}_{12} = 7$. Since $\bar{x}^{12}$ is integer $\bar{z}^{12} = z^{12} = 7$.
- Updating the incumbent. As the solution of $LP(S_{12})$ is integer, we update the value of the best feasible solution found $\underline{z} \leftarrow \max\{\underline{z} = 0, 7\}$ and store the corresponding solution $(2, 1)$. S_{12} is now **pruned by optimality**.





- Choosing a Node. The node list now contains only S_{11} .
- Reoptimizing. $S_{11} = S \cap \{x : x_1 \leq 2, x_2 \leq 0\}$. The resulting linear program has optimal solution $\bar{x}^{11} = (3/2, 0)$ and $\bar{z}^{11} = 6$. Since $\underline{z} = 7 > \bar{x}^{11} = 6$ the node is **pruned by bound**.
- Choosing a node. As the node list is empty, the algorithm terminates. The incumbent solution $x = (2, 1)$ with $z = 7$ is optimal.

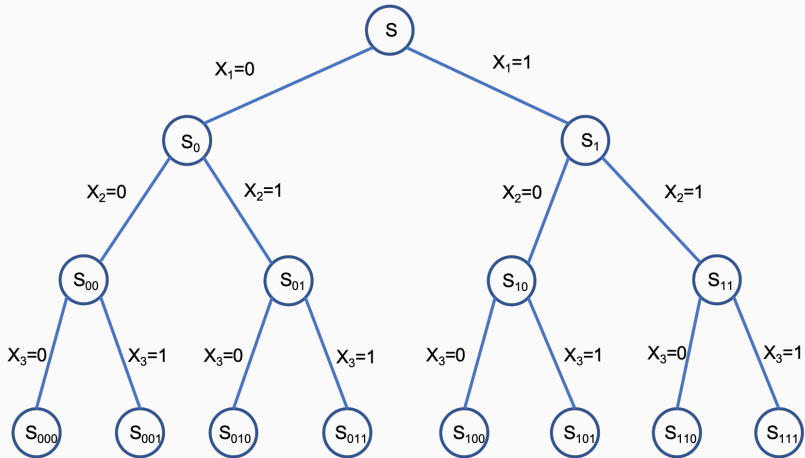
Pruning by optimality. $z^t = \max\{cx : x \in S_t\}$ has been solved.

Pruning by bound. $\bar{z}^t \leq \underline{z}$.

Pruning by infeasibility. $S_t = \emptyset$.

When addressing a binary problem, the natural choice is to branch on the binary variables, i.e., considering at each level of the tree a variable x_j and branching on it: $x_j = 0$ and $x_j = 1$.

Binary tree



The Knapsack problem is formulated as

$$\begin{aligned} z = \max \quad & \sum_{j=1}^n p_j x_j \\ & \sum_{j=1}^n w_j x_j \leq W \\ & x_j \in \{0, 1\} \text{ for } j = 1, \dots, n. \end{aligned}$$

We always assume $\sum_{j=1}^n w_j > W$. Otherwise the solution is trivial.

1. Variables are ordered so that $p_1/w_1 \geq \dots \geq p_n/w_n$.
2. Set

$$x_j = 1 \text{ for } j = 1, \dots, r-1$$

$$x_r = \frac{W - \sum_{j=1}^{r-1} w_j}{w_r}$$

$$x_j = 0 \text{ for } j = r+1, \dots, n$$

3. where r is such that $\sum_{j=1}^{r-1} w_j \leq W$ and $\sum_{j=1}^r w_j > W$.
4. If $\overline{W} = W - \sum_{j=1}^{r-1} w_j$ then $UB = \lfloor \sum_{j=1}^{r-1} p_j + p_r \frac{\overline{W}}{w_r} \rfloor$
5. We can prove it always holds that $z^* \leq UB$.

$$z = \max 6x_1 + 8x_2 + 7x_3 + 5x_4$$

$$3x_1 + 2x_2 + 5x_3 + 5x_4 \leq 8$$

$$x_j \in \{0, 1\} \text{ for } j = 1, \dots, 4$$

We first re-arrange the variables so that

$$p_1/w_1 \geq p_2/w_2 \geq p_3/w_3 \geq p_4/w_4$$

$$z = \max 8x_1 + 6x_2 + 7x_3 + 5x_4$$

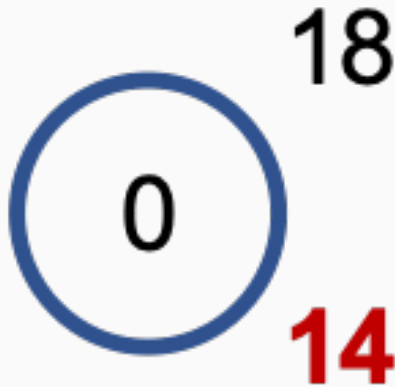
$$2x_1 + 3x_2 + 5x_3 + 5x_4 \leq 8$$

$$x_j \in \{0, 1\} \text{ for } j = 1, \dots, 4$$

We first calculate the upper bound UB_0

- Object 1 can enter, hence $x_1 = 1$ and $\overline{W} = 6$
- Object 2 can enter, hence $x_2 = 1$ and $\overline{W} = 3$
- Object 3 cannot enter, hence $r = 3$ and $x_3 = 3/5$
- $x_4 = 0$
- $UB_0 = \lfloor 8 + 6 + 7 * (3/5) \rfloor = \lfloor 14 + 21/5 \rfloor = 18$

A lower bound is easily identified by setting $x_1 = x_2 = 1, x_3 = x_4 = 0$, hence $LB = 14$



$$x_3 = 0$$

$$z = \max 8x_1 + 6x_2 + 5x_4$$

$$2x_1 + 3x_2 + 5x_4 \leq 8$$

$$x \in \{0, 1\}^4$$

- Object 1 can enter, hence $x_1 = 1$ and $\overline{W} = 6$
- Object 2 can enter, hence $x_2 = 1$ and $\overline{W} = 3$
- Object 4 cannot enter, hence $r = 4$ and $x_4 = 3/5$
- $UB_1 = \lfloor 8 + 6 + 5 * (3/5) \rfloor = \lfloor 14 + 3 \rfloor = 17$
- $LB_1 = 14$ due to $(1, 1, 0, 0)$

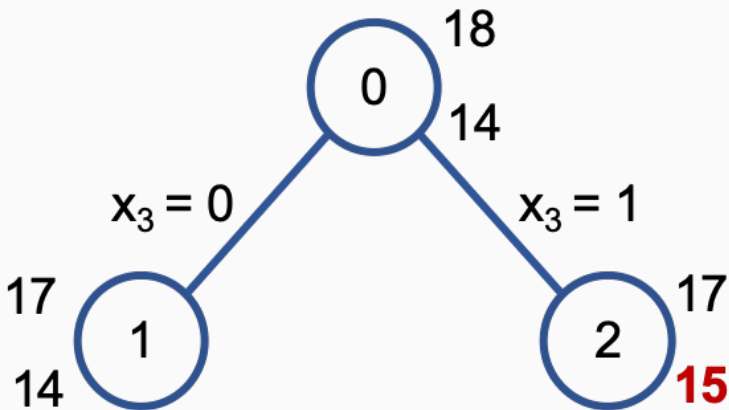
$$x_3 = 1$$

$$z = \max 8x_1 + 6x_2 + 5x_4 + 7$$

$$2x_1 + 3x_2 + 5x_4 \leq 3$$

$$x \in \{0, 1\}^4$$

- Object 1 can enter, hence $x_1 = 1$ and $\overline{W} = 1$
- Object 2 cannot enter, hence $r = 2$ and $x_2 = 1/3$
- $x_4 = 0$
- $UB_2 = \lfloor 8 + 7 + 6 * (1/3) \rfloor = \lfloor 15 + 2 \rfloor = 17$
- $LB_2 = 15$ due to $(1, 0, 1, 0)$
- Hence $LB = 15$



$x_3 = 0$, branching on x_4

$$x_3 = 0, x_4 = 0$$

$$z = \max 8x_1 + 6x_2$$

$$2x_1 + 3x_2 \leq 8$$

$$x \in \{0, 1\}^4$$

- Object 1 can enter, hence $x_1 = 1$ and $\overline{W} = 6$
- Object 2 can enter, hence $x_2 = 1$ and $\overline{W} = 3$
- $UB_3 = \lfloor 8 + 6 \rfloor = 14$
- $LB_3 = 14$ due to $(1, 1, 0, 0)$
- **STOP. Pruned by optimality.**

$$x_3 = 0, x_4 = 1$$

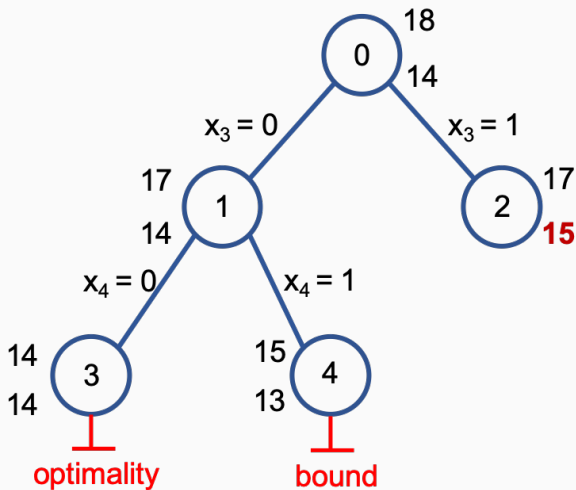
$$z = \max 8x_1 + 6x_2 + 5$$

$$2x_1 + 3x_2 \leq \mathbf{3}$$

$$x \in \{0, 1\}^4$$

- Object 1 can enter, hence $x_1 = 1$ and $\overline{W} = 1$
- Object 2 cannot enter, hence $r = 2$ and $x_2 = 1/3$
- $UB_4 = \lfloor 8 + 5 + 6 * (1/3) \rfloor = \lfloor 13 + 2 \rfloor = 15$
- $LB_4 = 13$ due to $(1, 0, 0, 1)$
- **STOP. Pruned by bound: $UB_4 = LB$**

$x_3 = 0$, branching on x_4



$x_3 = 1$, branching on x_2

$$x_3 = 1, x_2 = 0$$

$$z = \max 8x_1 + 5x_4 + 7$$

$$2x_1 + 5x_4 \leq 3$$

$$x \in \{0, 1\}^4$$

- Object 1 can enter, hence $x_1 = 1$ and $\overline{W} = 1$
- Object 4 cannot enter, hence $r = 4$ and $x_4 = 1/5$
- $UB_5 = \lfloor 8 + 7 + 5 * (1/5) \rfloor = \lfloor 15 + 1 \rfloor = 16$
- $LB_5 = 15$ due to $(1, 0, 1, 0)$

$$x_3 = 1, x_2 = 1$$

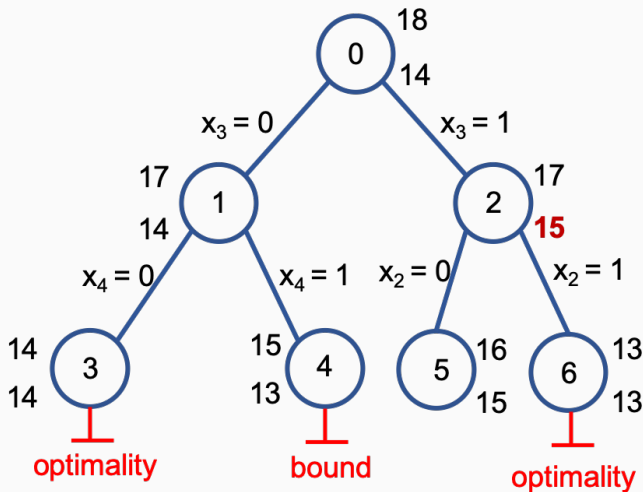
$$z = \max 8x_1 + 5x_4 + 13$$

$$2x_1 + 5x_4 \leq 0$$

$$x \in \{0, 1\}^4$$

- Object 1 cannot enter, $r = 1$ and $x_1 = 0$
- $x_4 = 0$
- $UB_6 = \lfloor 13 + 8 * (0) \rfloor = 13$
- $LB_6 = 13$ due to $(0, 1, 1, 0)$
- **STOP. Pruned by optimality.**

$x_3 = 1$, branching on x_2



$x_3 = 1, x_2 = 0$, branching on x_4

$$x_3 = 1, x_2 = 0, x_4 = 0$$

$$z = \max 8x_1 + 7$$

$$2x_1 \leq 3$$

$$x \in \{0, 1\}^4$$

- Object 1 can enter, hence $x_1 = 1$ and $\overline{W} = 1$
- $UB_7 = \lfloor 8 + 7 \rfloor = 15$
- $LB_7 = 15$ due to $(1, 0, 1, 0)$
- STOP. Pruned by optimality.

$$x_3 = 1, x_2 = 0, x_4 = 1$$

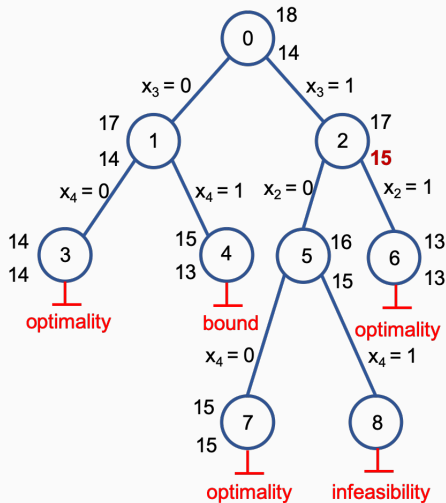
$$z = \max 8x_1 + 12$$

$$2x_1 \leq -2 \text{ INFEASIBLE!!}$$

$$x \in \{0, 1\}^4$$

- STOP. Pruned by infeasibility.

$x_3 = 1, x_2 = 0$, branching on x_4



- The optimal solution is $(1, 0, 1, 0)$ and $z^* = 15$
- This solution was found on node 2 but six other nodes had to be visited before confirming that $(1, 0, 1, 0)$ is indeed the optimal solution.

Thank you for your attention

